

Beauty Marketplace - Entrega 3 - Arquitetura

1. Identificação do grupo

Número do grupo: [preencher número do grupo]

Integrante 1: [nome completo] - RA [RA]

Integrante 2: [nome completo] - RA [RA]

Integrante 3: [nome completo] - RA [RA]

Integrante 4: [nome completo] - RA [RA]

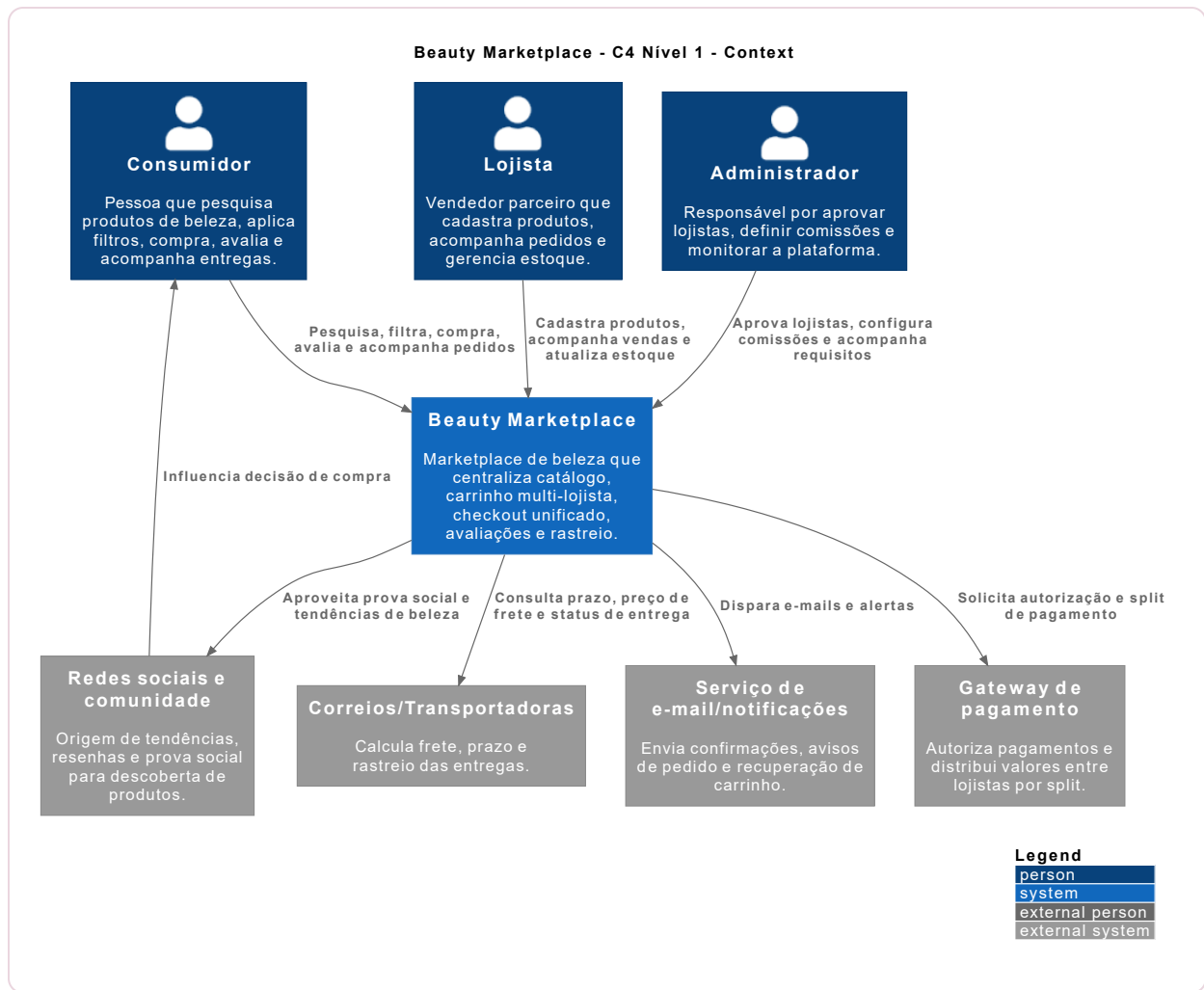
Objetivo

Apresentar a arquitetura completa do Beauty Marketplace, desde a visão de alto nível com C4 até os modelos de dados relacional e não relacional. A entrega utiliza C4-PlantUML como Diagrams as Code, MySQL para o modelo relacional e MongoDB/Redis para a modelagem NoSQL.

O Beauty Marketplace centraliza produtos de beleza de diferentes lojistas, com catálogo unificado, filtros por perfil de beleza, carrinho multi-lojista, checkout único, lista de desejos, avaliações e rastreamento por item.

2. Diagrama C4 - Nível Context (C1)

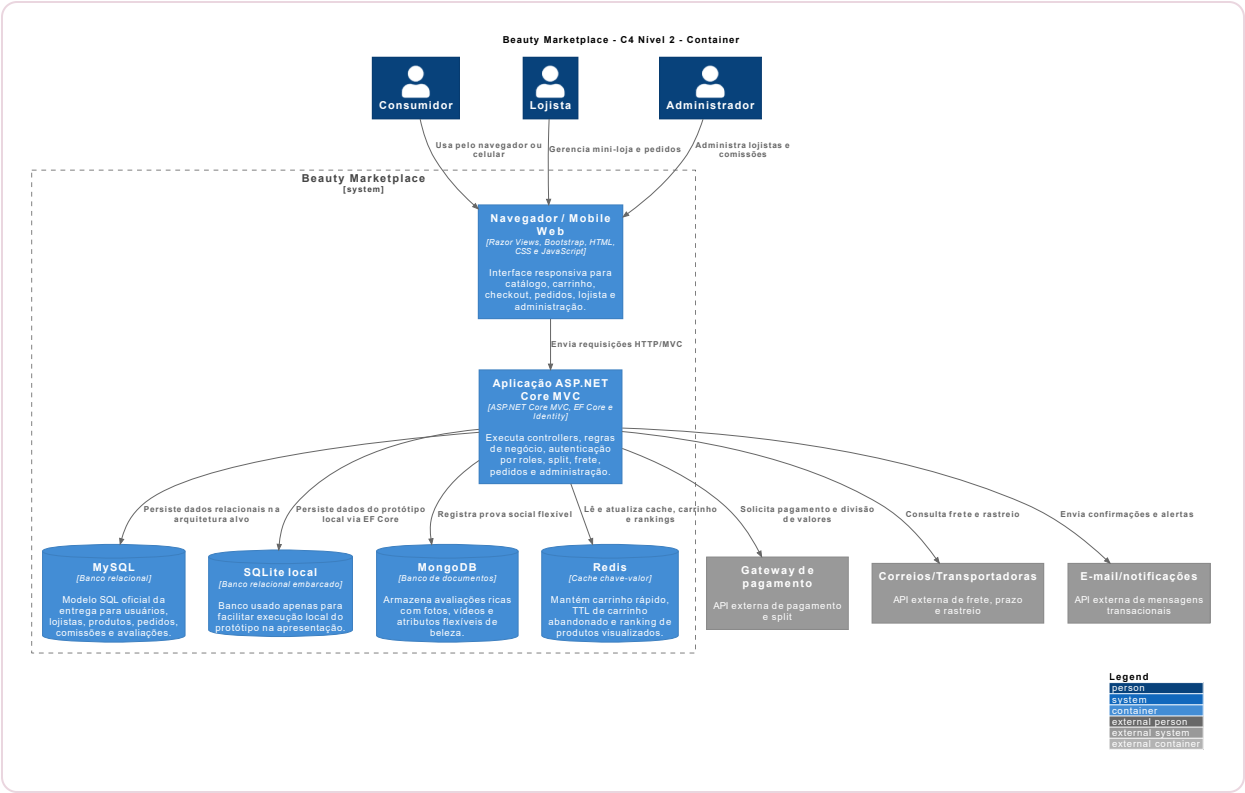
Arquivo fonte: `diagramas/c1-context.puml`



Elemento	Tipo	Descrição
Consumidor	Usuário	Pesquisa, filtra, compra produtos de beleza, avalia itens e acompanha entregas.
Lojista	Usuário	Cadastra produtos, acompanha vendas, gerencia estoque e recebe pedidos.
Administrador	Usuário	Aprova lojistas, define comissões e monitora a plataforma.
Beauty Marketplace	Sistema	Centraliza catálogo, carrinho multi-lojista, checkout, avaliações e rastreamento.
Gateway de pagamento	Sistema externo	Autoriza pagamento único e distribui valores entre lojistas por split.
Correios/Transportadoras	Sistema externo	Calcula preço de frete, prazo e status de rastreamento.
E-mail/notificações	Sistema externo	Envia confirmações, avisos e recuperação de carrinho abandonado.
Redes sociais e comunidade	Sistema externo/social	Origem de resenhas, tendências e prova social.

3. Diagrama C4 - Nível Container (C2)

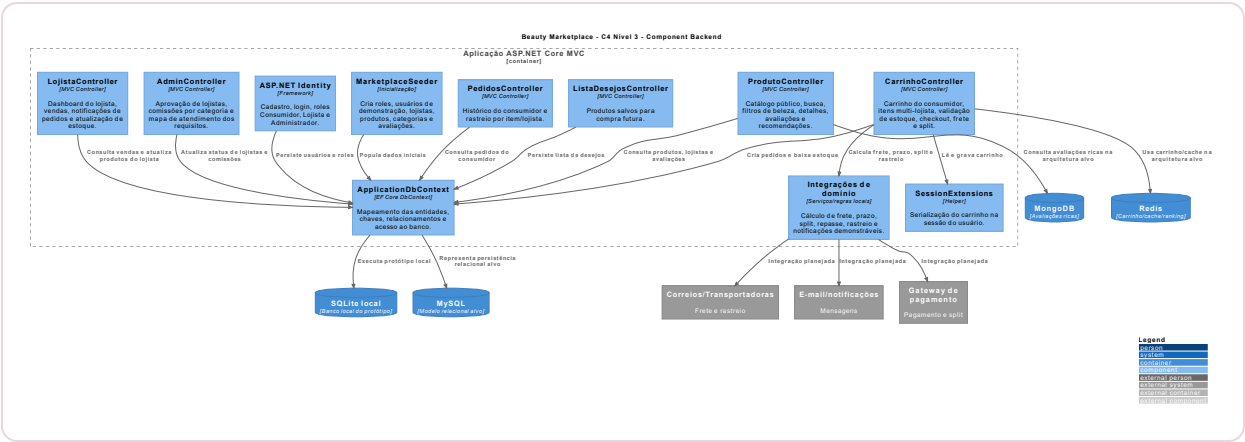
Arquivo fonte: `diagramas/c2-container.puml`



Container	Tecnologia	Justificativa
Navegador / Mobile Web	Razor Views, Bootstrap, HTML, CSS e JavaScript	Interface responsiva para demonstrar consumidor, lojista e administrador sem app nativo.
Aplicação ASP.NET Core MVC	ASP.NET Core MVC, EF Core e Identity	Base do projeto, com MVC, autenticação por roles e regras de marketplace.
MySQL	Banco relacional	Modelo oficial para dados transacionais, integridade referencial e consultas relacionais.
SQLite local	Banco relacional embarcado	Facilita execução local do protótipo, sem substituir o MySQL arquitetural.
MongoDB	Banco de documentos	Avaliações com mídias e atributos de beleza flexíveis.
Redis	Cache chave-valor	Carrinho rápido, TTL de abandono e ranking de produtos visualizados.
Serviços externos	APIs de pagamento, frete e notificações	Suportam split, prazo/rastreo e comunicação transacional.

4. Diagrama C4 - Nível Component (C3)

Arquivo fonte: `diagramas/c3-component-backend.puml`



Componente	Responsabilidade
ProdutoController	Catálogo, busca, filtros de beleza, detalhes, avaliações e recomendações.
CarrinhoController	Carrinho multi-lojista, estoque, checkout, frete, split e rastreo.
PedidosController	Histórico e rastreamento por item/lojista.
ListaDesejosController	Produtos salvos para compra futura.
LojistaController	Dashboard, vendas, notificações, cadastro/edição de produtos, upload de imagem e estoque do lojista.
AdminController	Aprovação de lojistas, comissões e mapa de atendimento.
ASP.NET Identity	Login, cadastro e roles Consumidor, Lojista e Administrador.
ApplicationDbContext	Mapeamento relacional e acesso ao banco via EF Core.

5. Diagrams as Code

Os diagramas C4 foram criados com PlantUML e C4-PlantUML. Os arquivos fonte ficam versionados no GitHub em `docs/Entrega 3/diagramas`.

```
plantuml "docs/Entrega 3/diagramas/*.puml"
```

6. Modelagem de dados - SQL (MySQL)

Arquivo fonte: `sql/mysql-schema.sql`

O modelo relacional possui 9 tabelas com chaves primárias, estrangeiras, checks e índices: usuarios, lojistas, categorias, comissoes_categoria, produtos, pedidos, pedido_itens, avaliacoes e lista_desejos_itens. A tabela produtos possui slug único para seed/upsert e URLs estáveis.

Relacionamento	Descrição
usuarios -> pedidos	Um consumidor pode realizar muitos pedidos.
lojistas -> produtos	Um lojista vende muitos produtos, cadastrados manualmente no painel ou carregados por catálogo JSON.
categorias -> produtos	Uma categoria classifica muitos produtos.
categorias -> comissoes_categoria	Cada categoria possui percentual de comissão vigente.
pedidos -> pedido_itens	Um pedido possui itens separados por produto e lojista.
produtos -> avaliacoes	Um produto pode receber muitas avaliações.
usuarios/produtos -> lista_desejos_itens	Consumidores salvam produtos para compra futura.

7. Modelagem de dados - NoSQL

MongoDB

Arquivo fonte: `nosql/mongodb-avaliacoes.json`. A coleção `avaliacoes_produto` guarda avaliações ricas com produto, lojista, usuário, nota, comentário, mídias, atributos de beleza, compra verificada e moderação.

Redis

Arquivo fonte: `nosql/redis-estruturas.md`. As estruturas modeladas são `cart:{usuarioId}` como Hash com TTL, `ranking:produtos:visualizados` como Sorted Set e `cart:abandoned:{usuarioId}` para recuperação de carrinho.

Checklist final dos critérios

Critério	Atendimento
C4 completo	C1, C2 e C3 incluídos com descrições.
Diagrams as Code	PlantUML/C4-PlantUML versionado no repositório.
SQL MySQL	Script com 9 tabelas, PKs, FKs e índices.
NoSQL	MongoDB e Redis modelados com caso de uso e justificativa.

Forma de entrega: enviar este PDF e o link do repositório GitHub contendo a pasta `docs/Entrega 3`.